

혜성 (THE COMET)

영화 추천 웹 서비스

5조

김혜령, 이규성

순서

01 주요프롬프트

02 ChatGPT 사용전후 비교

03 AI 기능 구현 캡처본

04 시연

05 마무리 및 QNA

01 주요프롬프트 (1)

[SYSTEM_PROMPT_FILTER]

```
SYSTEM_PROMPT_FILTER = f"""
너는 영화 추천 서비스의 '필터 추출기'다.
사용자 메시지를 읽고 DB 검색을 위한 필터를 JSON으로만 반환한다.

[절대 규칙]
- 출력은 반드시 순수 JSON만. 마크다운/코드펜스/설명 문장 금지.
- 장르는 반드시 이 목록에서만 선택: [{GENRES}]
- 키는 정확히 아래 스키마만 사용한다.
```

자연어 입력을 '검색 필터(JSON)'로 변환하는 역할을 프롬프트로 고정

“출력 형식을 강제해 파싱 실패/환각을 최소화

```
[JSON 스키마]
{{
  "answer": "사용자에게 보여줄 짧은 안내(1~2문장)",
  "filters": {{
    "primary_genre_name": null 또는 "장르1",
    "genre_names": ["장르1", "장르2"],
    "exclude_genre_names": ["제외장르1"],
    "exclude_titles": ["제외할 영화제목1"],
    "keywords": ["키워드1", "키워드2"],
    "titles": ["사용자가 언급한 영화제목"],
    "min_vote": 0~10 숫자,
    "strict": true/false
  }}
}}
```

고정 스키마(filters)로 추출 → 그대로 DB 쿼리 조건으로 연결

01 주요프롬프트 (2)

[SYSTEM_PROMPT_REASONS] + payload

```
SYSTEM_PROMPT_REASONS = """
너는 영화 추천 이유를 '영화 리스트 기반'으로 작성하는 에디터다.

[입력]
- user_request: 사용자가 원하는 분위기/조건
- movies: 영화 리스트(각 항목에 tmdb_id, title, genres(optional), vote_average(optional), overview(optional))

[출력 규칙]
- 반드시 순수 JSON만 출력한다.
- key는 tmdb_id(문자열)로 하고, value는 추천 이유 한 문장.
- 추천 이유는 20~35자 내외, 스포일러 금지, 사용자 조건과 영화 특징을 연결.
- 영화 리스트에 없는 tmdb_id를 만들지 말 것.
- 값이 비어 있으면 안 됨.
```

영화 목록 기반으로 tmdb_id별 ‘한 문장 추천 이유’를 JSON으로 강제
스포일러/길이 제한으로 UI 깨짐과 품질 편차 최소화

```
[출력 형식]
{
  "reasons": {
    "123": "겨울 감성에 어울리는 포근한 로맨스입니다.",
    "456": "힐링 분위기와 잔잔한 여운이 좋은 영화입니다."
  }
}
```

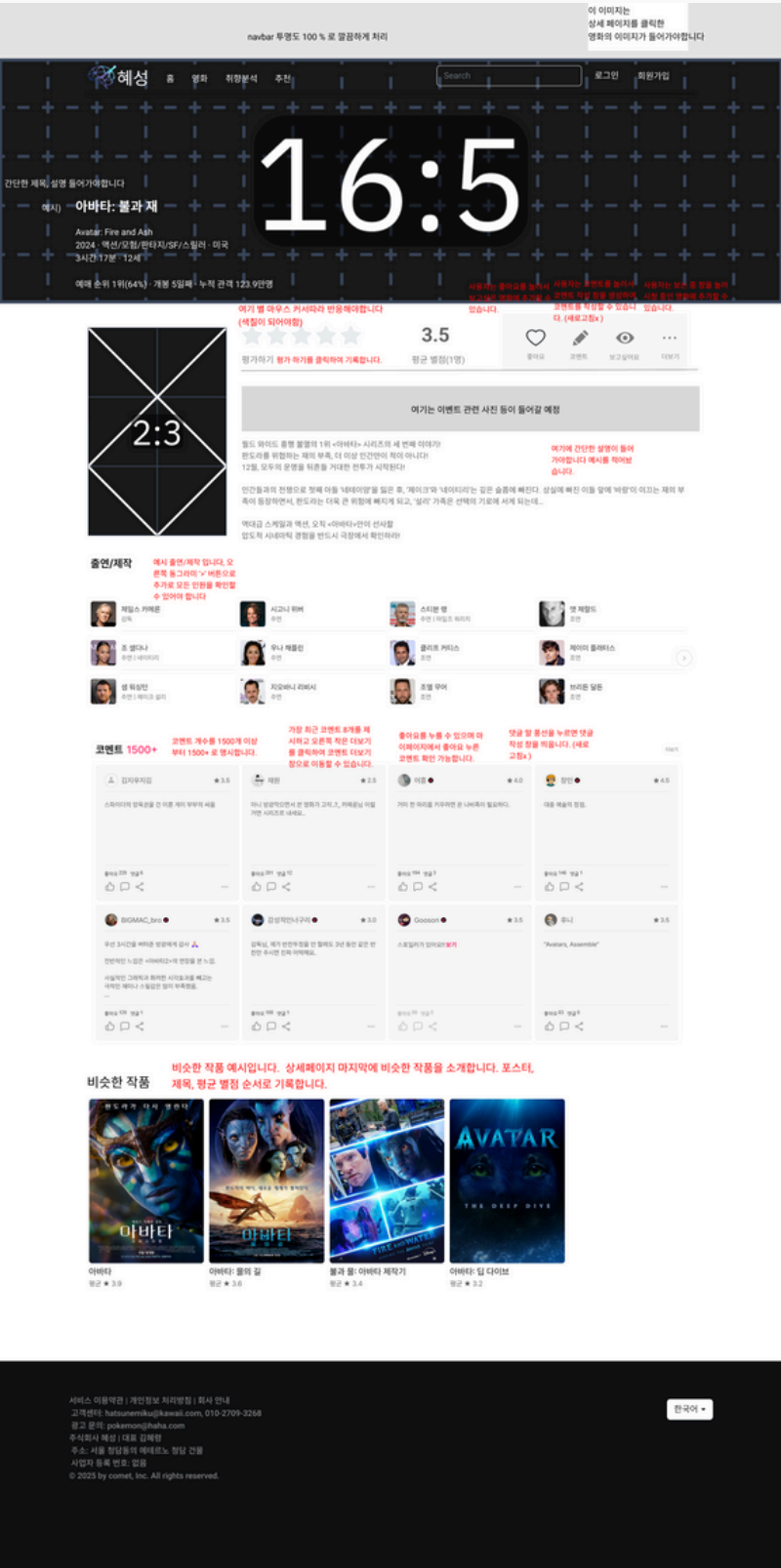
키를 tmdb_id로 고정해 프론트 카드에 이유를 안정적으로 매칭

```
def run_taste_ai_reasons(user_request: str, movies: List[Dict[str, Any]]) -> Tuple[Dict[str, str], str]:
    """
    DB에서 뽑힌 movies를 기반으로 tmdb_id별 추천 이유를 생성한다.
    movies 항목에는 최소 tmdb_id, title은 있어야 함.
    """
    payload = {
        "user_request": user_request,
        "movies": [
            {
                "tmdb_id": m.get("tmdb_id"),
                "title": m.get("title"),
                "genres": m.get("genres"),
                "vote_average": m.get("vote_average"),
                "overview": m.get("overview"),
            }
            for m in movies
        ]
    }
```

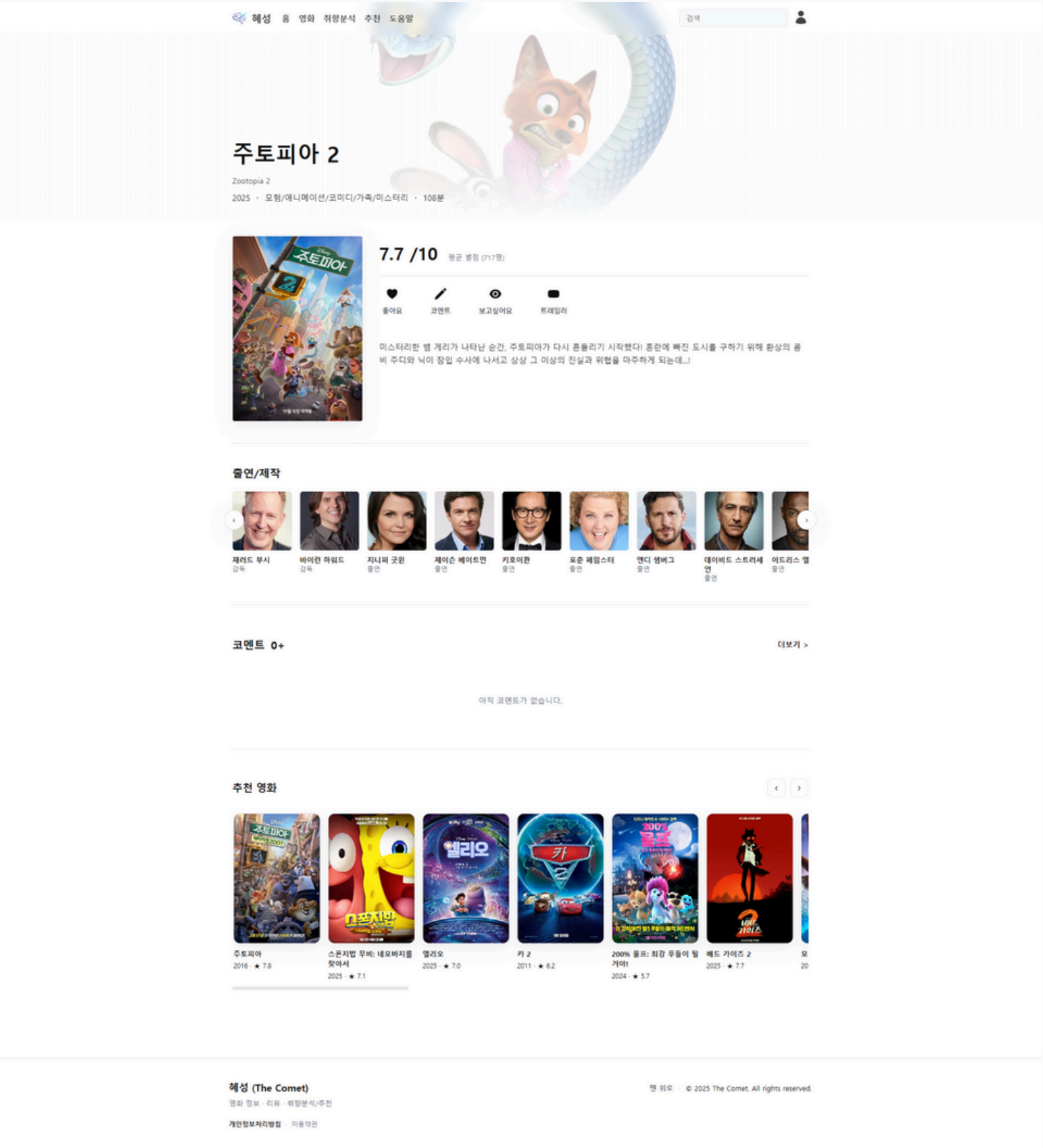
프롬프트에 ‘유저 요청 + DB에서 뽑은 영화 메타데이터’를 JSON으로 전달
LLM 출력(reasons)을 tmdb_id 기준으로 다시 붙여 UI에 표시

02 CHATGPT 사용전후 비교 (1)

Before 화면 구성/정보 구조(배치) 위주로 설계



After 실데이터 기반 섹션

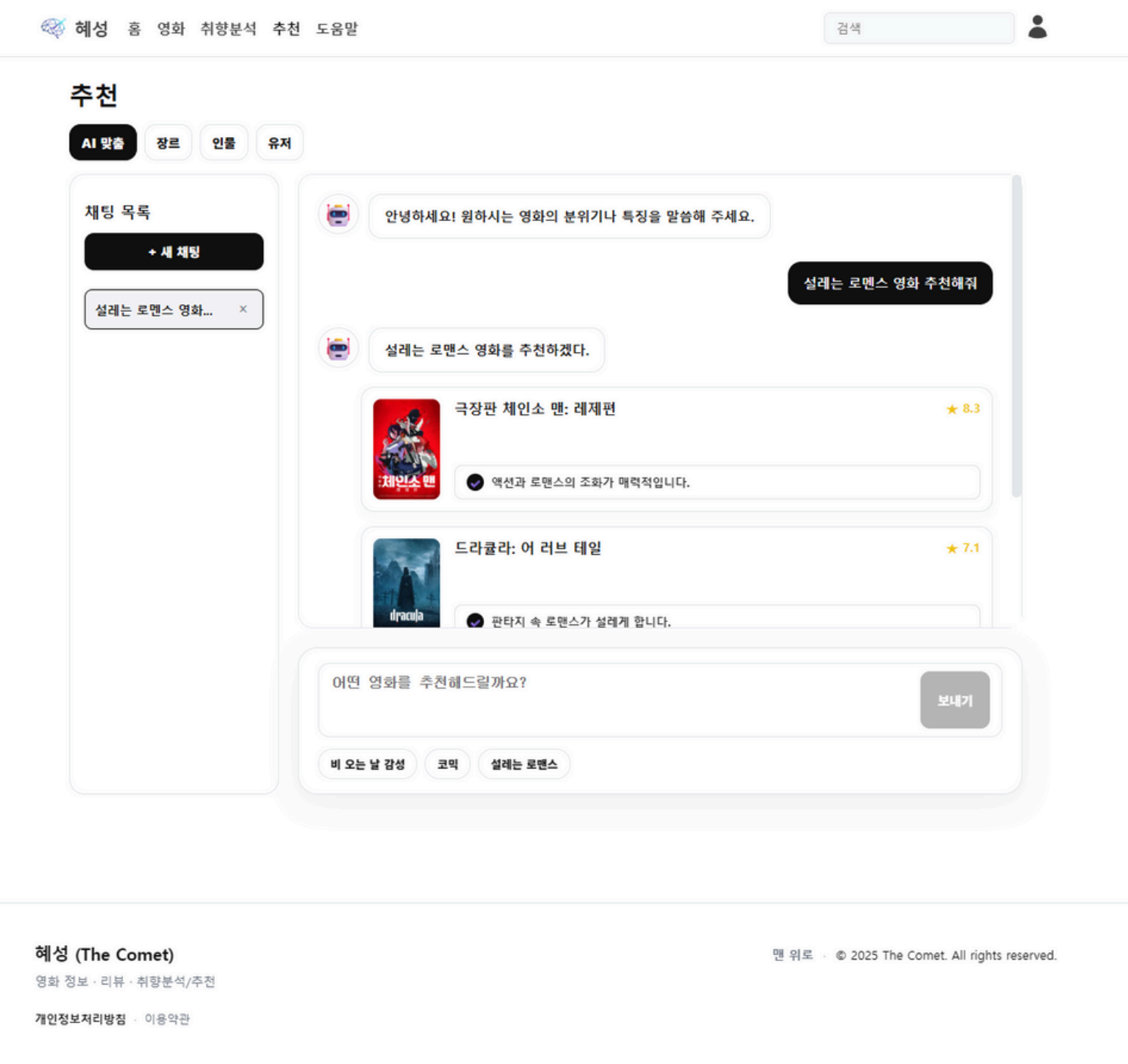


02 CHATGPT 사용전후 비교 (2)

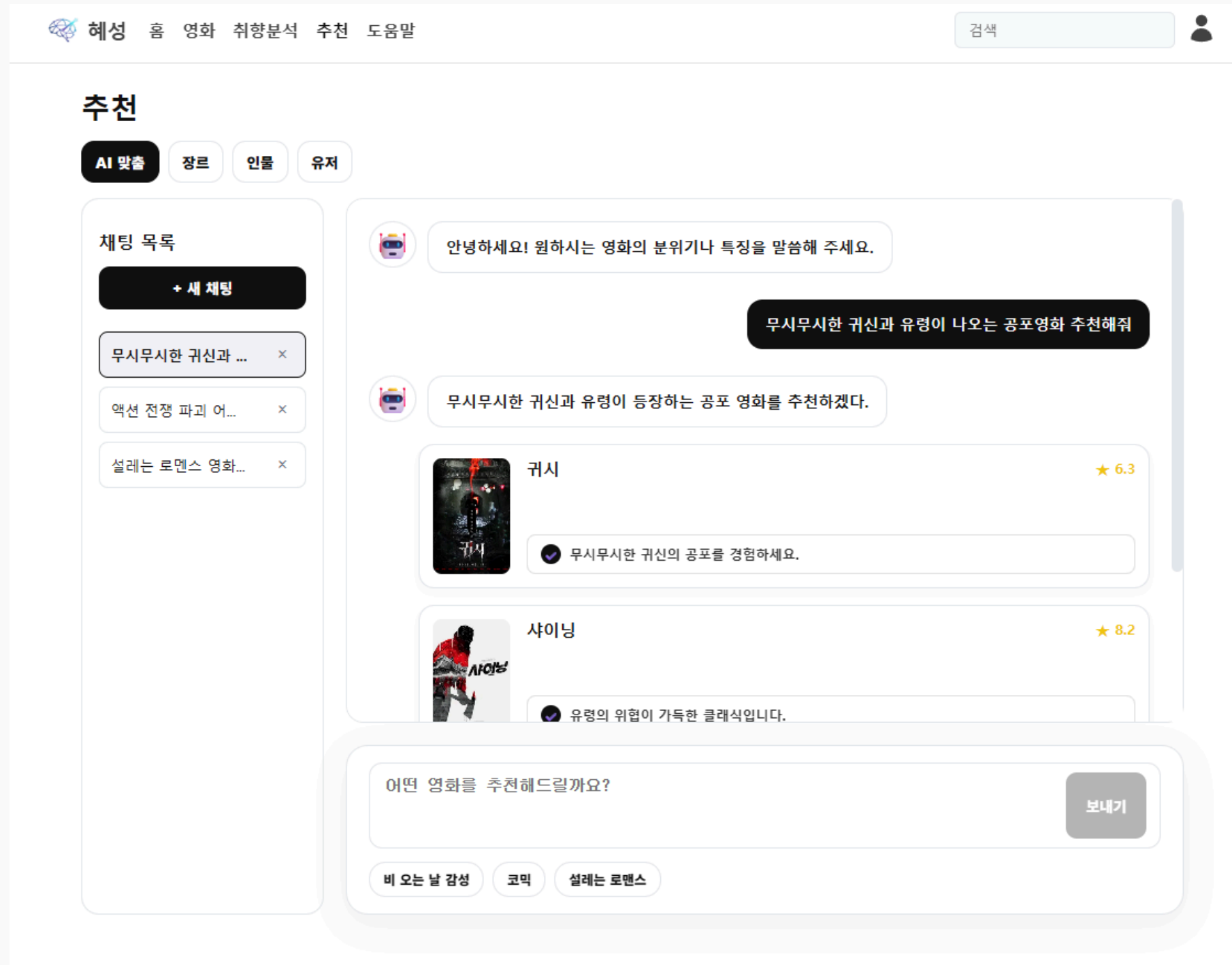
Before 채팅 UI + 추천 카드 “형태”만 먼저 정의



After 추천 근거/카드/대화 흐름까지 서비스화



03 AI 기능 구현 캡처본



04 시연 (LIVE DEMO)

(1) TMDb 전체 데이터 벌크 동기화 (페이지 단위)

```
python manage.py sync_tmdb_bulk --pages 25 --with-credits --sleep 0.35
```

(2) 홈 화면 구성용 데이터 동기화 (빠르게 갱신)

```
python manage.py sync_tmdb_home --pages 5 --no-credits
```

TMDb → DB로 사전 동기화하여, 화면 로딩 시 외부 API 의존도를 낮추고 일관된 추천/검색 경험을 제공합니다.

#시연 시나리오

1. 회원가입, 로그인 → 테마 변경(선택)
2. 홈 화면 기능
3. 영화 검색, 상세페이지 진입 → 토글, 리뷰작성, 좋아요 반영 확인
4. 리뷰 상세페이지 및 수정 삭제 확인
5. 마이페이지 → 보관함, 좋아요 연동 확인, 프로필 수정, 팔로잉/팔로워
6. 취향분석 페이지
7. 추천 페이지
8. footer 를 포함한 공통기능요소 소개

#핵심 포인트

- 토큰 만료 시 Refresh 재발급 + 요청 큐로 끊김 최소화
- AI 추천 이유는 tmdb_id 기준 매핑 → 카드에 안정적으로 표시
- 리뷰/댓글/좋아요는 즉시 상태 동기화(상세/마이페이지 연결)

05

마무리 및 QNA.

혹시 궁금하신 점 있으시면 편하게 질문해 주세요.

발표를 마치겠습니다.

발표를 들어주셔서 감사합니다.

5조 김혜령, 이규성